LAB 12 Cloud Firestore, Firebase Storage and push messaging

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Read/write authenticated cloud records in real time, upload an image to controlled cloud storage, and receive Firebase Cloud Messaging foreground events.

Core concepts. Firestore snapshots, document IDs and security rules; Firebase Storage object paths; image_picker; FCM token and foreground listener.

Before you start. Use the SAME Firebase project configured for Lab 11, but a separate Flutter demo project. Enable Anonymous sign-in for the exercises. FIREBASE CLOUD STORAGE currently requires a Blaze (pay-as-you-go) project; do not activate billing without instructor approval.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android --org edu.cui.sahiwal mad_lab12
cd mad_lab12
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage firebase_messaging
flutter pub add image_picker
dart pub global activate flutterfire_cli
flutterfire configure
```

Required Firebase console configuration and safe rules

Enable Authentication > Sign-in method > Anonymous for the demo. Create a Firestore database. Publish the following example rules in Firestore Rules so each anonymous signed-in user reads/writes only their own notes. For production, validate fields, sizes and app authorization more tightly. New anonymous identities may not survive app-data clearing.

FIRESTORE SECURITY RULES — Firebase console, not lib/main.dart

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/notes/{noteId} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }
  }
}
```

Solved task 1 | User-scoped Firestore notes with real-time updates

Problem and objective. Write notes to users/<uid>/notes and observe live database updates from StreamBuilder. Anonymous sign-in is enabled explicitly for this classroom demo.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  await FirebaseAuth.instance.signInAnonymously();
  runApp(const MaterialApp(home: CloudNotesPage()));
}

class CloudNotesPage extends StatefulWidget {
  const CloudNotesPage({super.key});
  @override
  State<CloudNotesPage> createState() => _CloudNotesPageState();
}

class _CloudNotesPageState extends State<CloudNotesPage> {
  final input = TextEditingController();
  @override
  void dispose() { input.dispose(); super.dispose(); }
```

```
  CollectionReference<Map<String, dynamic>> get notes {
    final uid = FirebaseAuth.instance.currentUser!.uid;
    return FirebaseFirestore.instance.collection('users')
      .doc(uid).collection('notes');
  }

  Future<void> add() async {
    final value = input.text.trim();
    if (value.isEmpty) return;
    try {
      await notes.add({'body': value,
        'createdAt': FieldValue.serverTimestamp()});
      input.clear();
    } catch (e) {
      if (mounted) ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Save failed: $e')));
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Firestore notes')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(12), child: Row(children: [
        Expanded(child: TextField(controller: input,
          decoration: const InputDecoration(labelText: 'Cloud note'))),
        IconButton(onPressed: add, icon: const Icon(Icons.cloud_upload)),
      ])),
      Expanded(child: StreamBuilder<QuerySnapshot<Map<String, dynamic>>>(
        stream: notes.snapshots(),
        builder: (context, snapshot) {
          if (snapshot.hasError) return Text('Firestore error: ${snapshot.error}');
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final docs = snapshot.data!.docs;
          return ListView.builder(itemCount: docs.length,
            itemBuilder: (context, i) {
              final row = docs[i];
              return ListTile(
                title: Text(row.data()['body'] as String? ?? ''),
                trailing: IconButton(icon: const Icon(Icons.delete),
                  onPressed: () => row.reference.delete()),
              );
            });
        },
      )),
    ]),
  );
}
```

How the solution works. The authenticated Firebase UID scopes the collection path. Firestore snapshots stream database changes to the app without manual polling. Each document has an auto-generated unique ID, and row.reference.delete() removes the selected document. Security is enforced by the Firestore Rules on the backend, not by hiding UI elements.

Expected output. Notes added from the phone appear in Firestore console and disappear live when deleted.

Check your work. Verify a second independently signed-in test user cannot read the first user's notes under the published rules.

Cloud Storage security rules and billing gate

Cloud Storage currently requires the Firebase Blaze (pay-as-you-go) plan. Before attempting Task 2, obtain instructor approval, enable billing/budget alerts, create the default bucket and then re-run flutterfire configure so lib/firebase_options.dart reflects the bucket. Publish the scoped rules below in Cloud Storage Rules. Never enable broad public write permissions. For production systems, also validate file size and content type.

CLOUD STORAGE SECURITY RULES — Firebase console

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /uploads/{uid}/{fileName} {
      allow read, write: if request.auth != null
        && request.auth.uid == uid;
    }
  }
```

```
}
```

Solved task 2 | Upload an image and inspect an FCM token/message

Problem and objective. Use image_picker to choose one gallery image, upload it under an authenticated user-scoped path and show its URL-based preview. Also request FCM permission, display the device token and listen for foreground messages. Storage upload requires the billing/rules gate above.

lib/main.dart

```
import 'dart:async';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_storage/firebase_storage.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:image_picker/image_picker.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  await FirebaseAuth.instance.signInAnonymously();
  runApp(const MaterialApp(home: MediaPage()));
}

class MediaPage extends StatefulWidget {
  const MediaPage({super.key});
  @override
  State<MediaPage> createState() => _MediaPageState();
}

class _MediaPageState extends State<MediaPage> {
  String status = 'No image selected';
  String? imageUrl;
  String token = 'Loading FCM token...';
  String message = 'No foreground push received';
  StreamSubscription<RemoteMessage>? subscription;

  @override
  void initState() { super.initState(); initMessaging(); }

  Future<void> initMessaging() async {
    try {
      final messaging = FirebaseMessaging.instance;
      await messaging.requestPermission();
      final value = await messaging.getToken();
      subscription = FirebaseMessaging.onMessage.listen((event) {
        if (mounted) setState(() => message =
          event.notification?.title ?? 'Data message received');
      });
      if (mounted) setState(() => token = value ?? 'No token yet');
    } catch (e) {
      if (mounted) setState(() => token = 'FCM unavailable: $e');
    }
  }

  Future<void> upload() async {
    final photo = await ImagePicker().pickImage(source: ImageSource.gallery);
    if (photo == null) return;
    setState(() => status = 'Uploading...');
    try {
      final uid = FirebaseAuth.instance.currentUser!.uid;
      final ref = FirebaseStorage.instance.ref()
        .child('uploads/$uid/${DateTime.now().millisecondsSinceEpoch}.jpg');
      await ref.putFile(File(photo.path));
      final url = await ref.getDownloadURL();
      if (mounted) setState(() {
        imageUrl = url; status = 'Upload complete';
      });
    } catch (e) {
      if (mounted) setState(() => status = 'Upload error: $e');
    }
  }

  @override
  void dispose() { subscription?.cancel(); super.dispose(); }
```

```
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Storage + FCM demo')),
    body: ListView(padding: const EdgeInsets.all(16), children: [
      FilledButton(onPressed: upload, child: const Text('Choose & upload image')),
      Text(status),
      if (imageUrl != null) Image.network(imageUrl!, height: 170),
      const Divider(),
      const Text('FCM device token (do not publish publicly):'),
      SelectableText(token, maxLines: 4),
      Text('Foreground notification: $message'),
    ]),
  );
}
```

How the solution works. image_picker supplies an XFile; putFile uploads a local image to a Firebase Storage object under uploads/<uid>/. This demonstration always writes a .jpg object name, so use a JPEG input or extend the example to preserve the real filename and MIME type before production use. A Firebase download URL can act as a shareable access token: do not paste it into public submissions. For FCM, getToken identifies this app instance and onMessage listens for foreground messages; treat tokens as sensitive test identifiers, never publish them.

Expected output. With Storage provisioned, an uploaded image appears and previews. On an FCM-capable phone, a token appears and a test foreground notification updates the label.

Check your work. Send a test from Firebase Console Cloud Messaging to a controlled test token; never store server credentials inside the Flutter application.

FCM delivery depends on Google Play services, Android notification permissions, app foreground/background behavior and the project's messaging configuration. A Flutter client alone cannot safely send administrative pushes: use Firebase Console for tests or a trusted backend using FCM HTTP v1 with authorized server credentials. Firestore does not itself send FCM messages.

Scenario-based practice — unsolved

Scenario 1. Group photo noticeboard. Users may only upload their own class images; staff can moderate and delete violating uploads. Define rules, object ownership, metadata validation and retention.

Scenario 2. Announcement notifications. Students subscribe to department announcements but must not see another department's private notices. Specify topic strategy, authenticated backend authorization, consent and token rotation.

Technical viva — justify your reasoning

1. Why are Firestore Rules more important than client-side hiding?
2. What is the difference between a document ID and a server timestamp?
3. Why does a streamed query usually not need polling?
4. What security risk follows globally readable Cloud Storage rules?
5. Why must FCM send credentials live only on a trusted backend?
6. What can prevent a device from obtaining an FCM token even when the Flutter code is correct?

Submission and self-check

Submit Firestore CRUD/stream evidence and rules; for Storage, show upload only if billing was approved. For FCM, redact token screenshots. Explain all external configuration steps.

CSC303 / NOVEMBER 2026 REVISION

LAB 12 EXTENSION | Safe cloud setup, permissions and fallback

Corrected package commands (supersede any split installation command in Lab 12):

```
flutter pub add firebase_core firebase_auth cloud_firestore
flutter pub add firebase_storage firebase_messaging image_picker
dart pub global activate flutterfire_cli
flutterfire configure
```

Run the commands from the `mad_lab12` app root, after creating it. Ensure the Dart global executable directory is on PATH before running `flutterfire`. The generated Firebase options are project/platform specific.

Instructor-only billing gate

Cloud Storage for Firebase requires Blaze billing. The instructor must approve, provision and monitor this service. A cost budget alert is a warning mechanism, not a guaranteed hard spending cap. Do not require students to enter payment methods for assessed work. Official reference: https://firebase.google.com/docs/storage/flutter/start.

Replace the example Storage rules with bounded writes

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /uploads/{uid}/{fileName} {
      allow read, delete: if request.auth != null
        && request.auth.uid == uid;
      allow create, update: if request.auth != null
        && request.auth.uid == uid
        && request.resource.size < 5 * 1024 * 1024
        && request.resource.contentType
             .matches('image/(jpeg|png|webp)');
    }
  }
}
```

Use a filename/extension matching the selected actual image type and provide consistent upload content-type metadata. The original example labels every upload .jpg even when a gallery picker may return a different format. Storage download URLs may be bearer links; do not publish them.

No-billing alternative and FCM testing

• If billing is not authorized: complete Task 1 Firestore scoped notes, then use an on-device image picker preview without upload. Record cloud upload as BLOCKED by policy, not failed.

• For FCM on Android 13+, request notification permission as needed, test foreground delivery from Firebase Console and redact registration tokens. A client token alone does not prove push delivery.

• Test a second anonymous user and verify that Firestore/Storage paths are isolated by UID. Rules, not widgets, enforce authorization.